Entrega técnica

Desenvolvimento 2

Entrega de Desenvolvimento 2

Decisões compartilhadas pelas APIs

Relatório técnico

Atualização sempre parcial com PATCH

Optei por `PATCH` porque as telas alteram campos específicos de um recurso, sem reenviar sua representação completa. Na PG API, por exemplo, `PATCH /api/v1/ciclo` recebe um objeto com os campos desejados: `{"titulo":"Novo título"}`, por exemplo. O service carrega o ciclo da empresa do usuário, valida se ele ainda pode ser alterado, modifica apenas os campos presentes no corpo e salva a entidade. Campos omitidos permanecem como estavam. CicloController · CicloService. O corpo é validado antes da alteração. Cada service define quais campos existem e quais podem ser modificados; `Validador.validarCampos` rejeita nomes desconhecidos e campos imutáveis, inclusive esse campo. No ciclo, por exemplo, `titulo` e `descricao` podem mudar, mas empresa e gestor não são aceitos no `PATCH` comum. Mudanças com regras próprias usam operações específicas, como `PATCH /api/v1/ciclo/status`, que verifica a transição e, no encerramento, consulta as regras do PostgreSQL. PatchConfig · Validador · CicloService.

A Mongo API segue a mesma ideia, com permissões próprias para cada documento. Na resposta de formulário, só `respostas` pode ser alterado; empresa, ciclo, formulário, usuário e data de envio continuam vinculados ao registro original. As novas respostas passam novamente pela validação conforme as perguntas do formulário antes de serem salvas. RespostaFormularioService.

Não implementei `PUT` porque ele representaria a substituição completa do recurso. Para os fluxos atuais, isso exigiria que o cliente enviasse também campos que não pretende mudar e aumentaria o risco de sobrescrever dados com uma representação incompleta. O `PATCH` expressa melhor a intenção de alterar poucos campos, enquanto as listas de campos permitidos e as regras do service protegem os dados controlados pelo servidor.

Separação entre controller, service, repository, DTO e mapper

As três APIs dividem a entrada HTTP, as regras de aplicação e a persistência. O **controller** declara rotas, recebe parâmetros e corpos validados e define a resposta HTTP. O **service** coordena regras, transações, autorização específica e integrações. O **repository** expressa consultas e operações do banco. O **DTO** define o contrato recebido ou devolvido ao cliente. O **mapper** converte esse contrato para entidade JPA ou documento MongoDB e de volta.

Entrega de Desenvolvimento 2

Decisões compartilhadas pelas APIs

Continuação · Separação entre controller, service, repository, DTO e mapper

Na PG API, `CicloController` delega ao `CicloService`, que usa `CicloRepository` e `CicloMapper`; na Mongo API, `RespostaFormularioService` acrescenta regras que não caberiam em um controller ou mapper, como conferir publicação, destinatário, perguntas e duplicidade. PG: controller, service, repository; Mongo: service de respostas.

Essa divisão reduz o acoplamento entre HTTP e banco: uma regra pode ser chamada e testada no service sem depender da montagem de uma resposta HTTP, enquanto uma mudança no formato do JSON pode ficar concentrada nos DTOs e mappers. Também impede que uma entidade persistida seja exposta automaticamente ao cliente, inclusive com campos internos ou relações que não fazem parte do contrato. O custo é criar mais classes e manter o mapeamento coerente. A base genérica de CRUD nas APIs PG e Mongo reaproveita a sequência comum de conversão, persistência e resposta, enquanto os serviços concretos implementam regras próprias. PG: `BaseService`, Mongo: `BaseService`.

O **MapStruct** gera a implementação dos mappers a partir de interfaces declarativas. Isso torna explícitos os campos que devem ser ignorados na entrada: `EmpresaMapper` não aceita que o DTO determine campo, auditoria, status e coleções administradas pelo domínio, e seu `@AfterMapping` liga endereços e contatos à empresa. `FormularioMapper` ignora empresa, ciclo, `status` e `publicadoEm`, que são atribuídos pelo service. Assim, a conversão não concede ao corpo da requisição autoridade sobre campos de contexto ou ciclo de vida. Mapeamentos especiais continuam expressos por `@Mapping`, métodos auxiliares ou `@AfterMapping`. PG: `EmpresaMapper`, Mongo: `FormularioMapper`, Auth: `AuthMapper`.

Por que usar record nos DTOs

Os contratos de entrada e saída são, em sua totalidade, `record`: `EmpresaRequestDTO`, `ConviteRequestDTO`, `FormularioRequestDTO` e `MeResponseDTO` são exemplos. O `record` declara uma estrutura de dados com construtor, métodos de acesso e igualdade por valor sem setters; isso combina com um objeto que representa o conteúdo de uma requisição ou resposta em um momento definido. As anotações de Bean Validation ficam nos componentes do contrato e a entidade persistida permanece uma classe mutável, adequada ao JPA ou ao MongoDB. É uma escolha de clareza de contrato e redução de código repetitivo, não uma regra de que todo objeto da aplicação deva ser imutável.

Entrega de Desenvolvimento 2

Decisões compartilhadas pelas APIs

Continuação · Por que usar `record` nos DTOs

Uma lista contida em um `record`, por exemplo, ainda pode ser mutável se não for copiada. PG: `EmpresaRequestDTO`, Auth: `ConviteRequestDTO`, Mongo: `FormularioRequestDTO`.

Mensagens de validação

Nos DTOs, anotações como `@NotBlank`, `@NotNull`, `@Size`, `@Email` e `@Valid` verificam o formato básico antes da regra de negócio. Em vez de repetir uma frase em cada campo, as APIs usam chaves como `{validation.empresa.cnpj.notblank}` e `{validation.formulario.titulo.notblank}`, resolvidas em `ValidationMessages.properties`. Isso padroniza a linguagem dos erros, facilita a manutenção do texto e deixa a regra declarada junto ao dado que ela valida. `@Valid` nas listas e objetos aninhados faz a validação percorrer os componentes internos. Já regras dependentes do estado persistido — por exemplo, formulário ativo ou convite pendente — continuam no service. PG: DTO e mensagens; Mongo: DTO e mensagens.

Clients para integração entre APIs e serviços externos

Um `client` concentra endereço, cabeçalhos, serialização, timeout e tradução de erro de outro serviço. Na PG API, `AuthApiClient` consulta `/api/v1/contexto`; `AuthApiFilter` transforma o contexto devolvido em principal Spring Security e diferencia falhas 401, 403 e indisponibilidade 503. Na Mongo API, `AuthApiClient` obtém esse mesmo contexto, enquanto `PgApiClient` consulta ciclos da PG API encaminhando o `Authorization` recebido. Isso evita replicar a validação Firebase e o cadastro relacional na Mongo API e deixa explícita a dependência entre serviços. Clients também existem para Brevo, FCM API e ViaCEP, cada qual isolando seu protocolo externo. PG: `AuthApiClient` e filtro; Mongo: `AuthApiClient` e `PgApiClient`.

Essa opção troca duplicação de lógica por dependência de rede: uma operação protegida da PG ou da Mongo API precisa que a Auth API responda. Os clients da Mongo API configuram tempos de conexão e leitura, e os filtros tratam indisponibilidade com 503. A separação deixa claro quem responde por cada dado, mas não elimina a necessidade de observar latência, disponibilidade e contratos entre APIs. Mongo: configuração, filtro de autenticação.

Entrega de Desenvolvimento 2

Decisões da PG API

Relatório técnico

Migração da autenticação para a Auth API

A PG API recebe um token do Firebase no cabeçalho Bearer, mas não valida diretamente esse token com o Firebase. Seu `AuthApiFilter` o envia à Auth API por `GET /api/v1/contexto`; só depois de receber usuário, empresa e papel válidos monta o principal da requisição. Na Auth API, `FirebaseAuthFilter` verifica o token e `FirebaseUtils` consulta o perfil relacional, recusando usuário, empresa ou colaborador inativo antes de devolver o contexto. Essa extração coloca a política de identidade e estado do perfil em um ponto único para PG e Mongo, mantendo a PG API concentrada nas operações PDCA. **PG: filtro**; **Auth: filtro Firebase**, `FirebaseUtils`.

Notificações publicadas após o commit

Alterações de tarefas e ciclos podem gerar eventos para a FCM API: atribuição, reatribuição, bloqueio, desbloqueio e mudança de status do ciclo. Os services escolhem o evento e os destinatários; `FcmApiClient` constrói o payload, inclui a referência à empresa, um deep link e uma chave de deduplicação, e envia `POST /internal/v1/eventos` com `X-Internal-Token`. **PG:** `TarefaService`, `CicloService`, `FcmApiClient`.

`FcmApiClient.publicarAposCommit` registra uma `TransactionSynchronization` quando há sincronização transacional e executa o envio em `afterCommit`. A decisão impede anunciar uma alteração que acabou sendo revertida no PostgreSQL. O envio é um callback síncrono após o commit. Se não houver sincronização ativa, o próprio client executa a publicação imediatamente.

Entrega de Desenvolvimento 2

Decisões da PG API

Continuação · Notificações publicadas após o commit

Depois de um commit bem-sucedido, falhas traduzidas para `BusinessRuleException` são ignoradas pelo callback (o cliente não precisa saber que o envio de notificação falhou na sua resposta); o dado permanece salvo e o evento pode se perder. Não há retentativa persistente nesse fluxo. A escolha prioriza a consistência da mensagem com o dado confirmado, aceitando uma janela de falha entre os dois sistemas. **PG:** `FcmApiClient`, **README**.

Regras em functions e procedures PostgreSQL

Algumas regras dependem de vários registros relacionais ou precisam acompanhar uma atualização no próprio banco. `CicloRepository` chama `pdca.fn_avanco_ciclo`, `pdca.fn_pode_encerrar_ciclo` e `pdca.pr_encerrar_ciclo`; `TarefaRepository` chama `pdca.fn_pode_iniciar_tarefa` e `pdca.pr_reabrir_tarefa`.
O service ainda controla autorização, status e resposta HTTP, mas consulta a regra SQL antes da transição ou executa a procedure dentro da transação. **PG:** `CicloRepository`, `CicloService`, `TarefaRepository`, `TarefaService`.

O **SQL versionado em** `acta-database` mostra o conteúdo dessas rotinas: o avanço calcula percentual de tarefas concluídas ou canceladas; o encerramento rejeita ciclos inexistentes, já encerrados ou com planos e tarefas pendentes; o início de tarefa verifica vínculo ao ciclo, plano em execução, responsabilidade, treinamento obrigatório e dependências; a reabertura exige tarefa concluída ou cancelada e novo prazo válido. Centralizar essas verificações no PostgreSQL permite avaliar os dados relacionais perto de onde são persistidos e reaproveitar a regra em operações do banco. Em contrapartida, a aplicação passa a depender da versão do esquema e das rotinas SQL.

Entrega de Desenvolvimento 2

Decisões da Auth API

Relatório técnico

Firebase como identidade; PostgreSQL como perfil de negócio

O cliente faz login no Firebase e envia um token do Firebase; a Auth API não implementa senha local nem emite sessão própria. FirebaseUtils.verificarToken usa o Firebase Admin SDK com verificação de revogação. A identidade Firebase do token é procurada na tabela relacional `usuario_sistema`. Nela ficam empresa, papel, status e vínculo com colaborador, dados que determinam o acesso ao ACTA. A separação permite deixar credenciais e prova de identidade com o provedor, enquanto o projeto controla as regras próprias do negócio no PostgreSQL. **Auth:** `FirebaseUtils`, `Usuario`, `SecurityConfig`.

Dois estados de acesso

Uma identidade Firebase válida ainda pode não ter perfil. O filtro concede `ROLE_FIREBASE` às rotas `/me`, ativação e onboarding, nas quais basta a identidade verificada, ou seja, apenas ter um cadastro no Firebase (que não significa que o usuário realmente está ativo). `GET /api/v1/me` consulta a identidade e devolve `CADASTRO_NAO_INICIADO` quando não há usuário relacional; quando encontra, devolve `PERFIL_CRIADO` e os dados do perfil. Para as outras rotas, o filtro exige um usuário encontrado e `FirebaseUtils.validarAcesso` exige usuário e empresa ativos, além de colaborador ativo quando associado. Assim, a aplicação consegue orientar cadastro ou ativação sem conceder acesso prematuro às operações de negócio. `PERFIL_CRIADO` informa a existência do registro; o acesso pleno ainda depende dos status ativos verificados nas rotas protegidas. **Auth:** `FirebaseAuthFilter`, `AuthService`, `FirebaseUtils`, `MeResponseDTO`.

Onboarding e primeiro ADMIN

O primeiro administrador não pode depender de um convite emitido por outro administrador da própria empresa. Por isso, `OnboardingService.iniciar` aceita `ROLE_FIREBASE`, exige e-mail verificado igual ao e-mail do gestor, verifica duplicidade de e-mail e CPF e valida o CNPJ. Se a empresa já existe, orienta procurar um `ADMIN` ativo para receber convite. Se não existe, `cadastrarEmpresa` cria empresa, usuário `ADMIN` e colaborador vinculados, com status ativo e vínculo Firebase associado. O fluxo é transacional;

Entrega de Desenvolvimento 2

Decisões da Auth API

Continuação · Onboarding e primeiro `ADMIN`

Essa regra evita que um segundo usuário se atribua a administração de uma empresa já cadastrada apenas por informar seu CNPJ. **Auth:** `OnboardingService`, `OnboardingController`.

Brevo e convite com código de uso controlado

Um `ADMIN` da empresa cria um usuário e colaborador pendentes, gera um código de seis caracteres com `SecureRandom` e persiste apenas seu hash SHA-256 no registro de convite. O código em claro segue no e-mail montado a partir de um template HTML e enviado pelo `BrevoClient`. O convite expira em 30 minutos e possui estados `PENDENTE`, `USADO` e `REVOGADO`. Para ativar, a pessoa apresenta o código junto a uma identidade Firebase com e-mail verificado; a Auth API compara o hash, o prazo, o status e o e-mail do token com o destino e o login. Na mesma transação, vincula a identidade, ativa usuário e colaborador e marca o convite como usado. O reenvio revoga convites pendentes válidos antes de gerar outro. **Auth:** `ConviteService`, `TokenUtils`, `Convite`, `BrevoClient`. **Auth:** `ConviteService`, `TokenUtils`.

Autorização por papel e empresa no service

As rotas protegidas exigem uma identidade e um perfil ativos, mas algumas operações têm critérios mais específicos. ConviteService.convidar exige o papel ADMIN e valida o vínculo com a empresa; `reenviar` combina o mesmo papel com a verificação de que o colaborador pertence à empresa do administrador. `AuthService` faz essas consultas pela empresa do principal. A regra fica junto à operação de negócio que protege, inclusive se ela vier a ser chamada por outro controller, e impede que um administrador de uma empresa atue por dados de outra. A PG API também aplica filtros por empresa em consultas filtradas por empresa, mostrando que autorização de rota e escopo de dados são verificações complementares. **Auth:** `ConviteService` e `AuthService`; **PG:** `CicloService`.

Entrega de Desenvolvimento 2

Decisões da Mongo API

Relatório técnico

Strategy para respostas de tipos diferentes

Um formulário contém perguntas com tipos distintos. `RespostaFormularioValidator` recebe do Spring uma lista de implementações de `ValidacaoRespostaStrategy`, organiza um `EnumMap<TipoResposta, ValidacaoRespostaStrategy>` e, para cada resposta, procura a estratégia correspondente ao tipo definido na pergunta armazenada. Antes disso, rejeita pergunta repetida, pergunta que não pertence ao formulário, resposta nula e ausência de pergunta obrigatória. A estratégia então verifica o formato específico: `RADIO` exige uma opção da pergunta, `RANGE` verifica faixa numérica, `CEP` exige oito dígitos e prepara um endereço via ViaCEP. Desse modo, uma nova validação pode ser isolada em uma classe, sem ampliar um único bloco de condicionais no service. **Mongo:** `RespostaFormularioValidator`, `ValidacaoRespostaStrategy`, `ValidacaoRadioStrategy`, `ValidacaoCepStrategy`. A seleção usa o **tipo da pergunta persistida**, e não um tipo escolhido pelo cliente no corpo da resposta. Após validar, `preparar` permite normalizar ou enriquecer o valor antes do mapeamento para o documento. `TipoResposta`, `RespostaFormularioValidator`, **estratégias versionadas**.

Referências na PG API e contexto na Auth API

Os documentos MongoDB guardam referências de empresa, ciclo e usuário que pertencem ao contexto relacional. A Auth API é consultada a cada requisição protegida para fornecer o usuário autenticado, empresa e authorities; o token original é encaminhado. Ao criar ou listar Ishikawa, formulários e lições aprendidas por ciclo, o service chama `PgApiClient.buscarCiclo` e compara empresa do ciclo com a empresa do principal. Em seguida, preenche empresa e ciclo do documento no servidor. Isso evita aceitar do corpo um vínculo arbitrário com um ciclo de outra empresa. Referências entre documentos Mongo também são verificadas no próprio domínio, como a causa pertencente ao Ishikawa informado em `CincoPorquesService`. **Mongo: filtro Auth**, `PgApiClient`, `IshikawaService`, `FormularioService`, `CincoPorquesService`. Essa conferência é feita pela aplicação por chamada HTTP, não por uma chave estrangeira entre MongoDB e PostgreSQL.

Entrega de Desenvolvimento 2

Decisões da Mongo API

Continuação · Referências na PG API e contexto na Auth API

Ela confirma existência e pertencimento no momento da consulta, mas não constitui transação distribuída que preserve automaticamente a referência se o ciclo for alterado ou removido depois. **Mongo:** `FormularioService`.

Uma resposta por usuário, inclusive sob concorrência

`RespostaFormularioService.inserir` exige formulário `ATIVO`, confere se o usuário está entre os destinatários quando a lista é restrita, usa o usuário do contexto autenticado e consulta se já existe resposta para aquela empresa, ciclo, formulário e usuário. Essa verificação antecipada permite devolver uma mensagem de conflito clara, mas duas requisições simultâneas ainda poderiam passar por ela antes de gravar. Por isso `RespostaFormulario` declara um índice composto único para impedir respostas duplicadas no mesmo contexto. Na tentativa de gravação, a exceção de chave duplicada desse índice é traduzida para `DuplicateFormResponseException`. A regra é, portanto, comunicada no service e imposta também pelo banco quando o índice estiver criado. **Mongo:** `RespostaFormularioService`, `RespostaFormulario`, `RespostaFormularioRepository`. O service também restringe a atualização parcial ao campo `respostas` e busca a resposta pela empresa autenticada; usuário, formulário e os demais campos de contexto não são aceitos no patch. . **Mongo:** `RespostaFormularioService`.